



Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

Agent Based Modelling- Lab examples and simulations

CSC1111
Building Complex Computational Models

Marija Bezbradica

Notes

- In labs today we have two sets of examples:
- One are marked as '**theoretical**' – here you are expected to **read** the instructions and explanations from slides and go through the code which is uploaded on your Loop page. You are expected to **understand** and run this code for different parameter values in the simulator.
- The other examples are marked as '**upload your solution on Loop**' – here you are expected to **adapt** the code from theoretical examples and to answer all the questions posted. For these examples you are expected to **upload** your solutions on your Loop page. Any comments would be welcomed. You should copy all your code, with comments in between, into one file and submit as one pdf file.

Example 9.1 (Sayama book) (theoretical)

For all our examples we will use Sayama book and *pycxsimulator.py* for ABM simulations (introduced in week 7 lectures and slides <https://github.com/hsayama/PyCX>)

Schelling's segregation model – From 1987 - Early model on why people with different ethnic backgrounds tend to segregate geographically. Nobel prices in Economics in 2005.

Here are the model assumptions:

- Two different types of agents are distributed in a finite 2-D space.
- In each iteration, a randomly chosen agent looks around its neighbourhood, and if the fraction of agents of the same type among its neighbors is below a threshold, it jumps to another location randomly chosen in the space.

Schelling mainly investigated threshold as to when segregation is happening. Initially, around 80% was assumed as critical value but he proved this to be much lower i.e. segregation can appear even if people are not so homophilic.

Regardless of model complexity we still need the same three components:

- **initialization, visualization, and updating functions.**

Example 9.1 –cont.

1. Design the data structure to store the attributes of the agents. In this model, each agent has a type attribute and 2D position. The two types can be represented by 0 and 1, and the spatial position can be anywhere within a unit square. Populate with agents.

```
n = 1000 # number of agents

class agent:
    pass

def initialize():
    global agents
    agents = []
    for i in range(n):
        ag = agent()
        ag.type = randint(2)
        ag.x = random()
        ag.y = random()
        agents.append(ag)
```

Example 9.1 –cont.

Design phase

1. Design the data structure to store the **attributes** of the agents
2. Design the data structure to store the **states** of the environment
3. Describe the **rules** how the **environment** behaves
4. Describe the **rules** for how agents **interact** with the environment
5. Describe the rules for how **agents** behave
6. Describe the rules for how agents **interact** with each other

✓ We can skip 2, 3, 4, 5 from previous slides as no separate environment and agents don't do anything by themselves, because their actions (movements) are triggered only by interactions with other agents

But, for 6. – Assumption is each agent checks who is in its neighbourhood, and if the fraction of the other agents of the same type is less than a threshold, it jumps to another randomly selected location. This requires neighbour detection (done in CA example but it was simpler there). ag - local agent

```
neighbors = [nb for nb in agents
              if (ag.x - nb.x)**2 + (ag.y - nb.y)**2 < r**2 and nb != ag]
```

Example 9.1 –cont.

- Simplest possible method: **Exhaustive search**.
- Check all the agents, one by one, to see if they are close enough to the focal agent. Computationally not efficient but easy implementation for now.
- Here ag is the focal agent whose neighbors are searched for. The if part in the list measures the distance squared between ag and nb, and if it is less than r squared nb is included in the result.

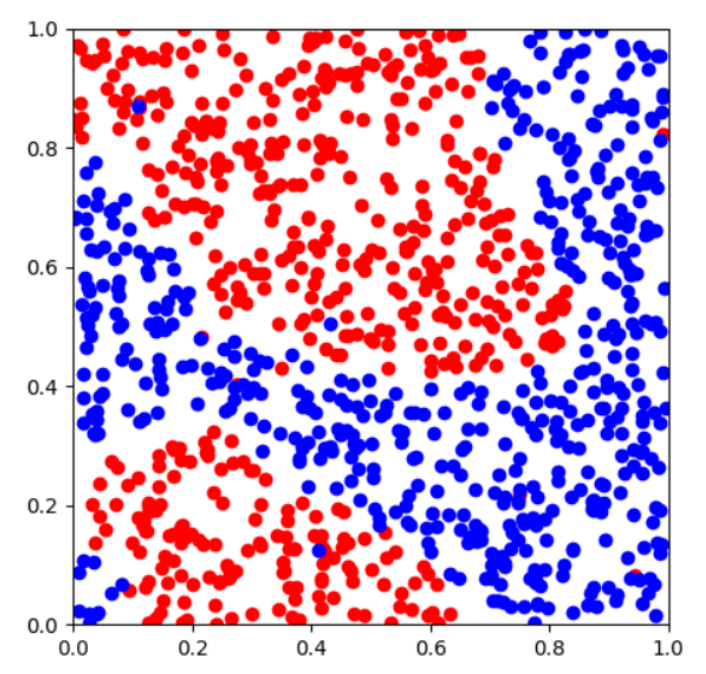
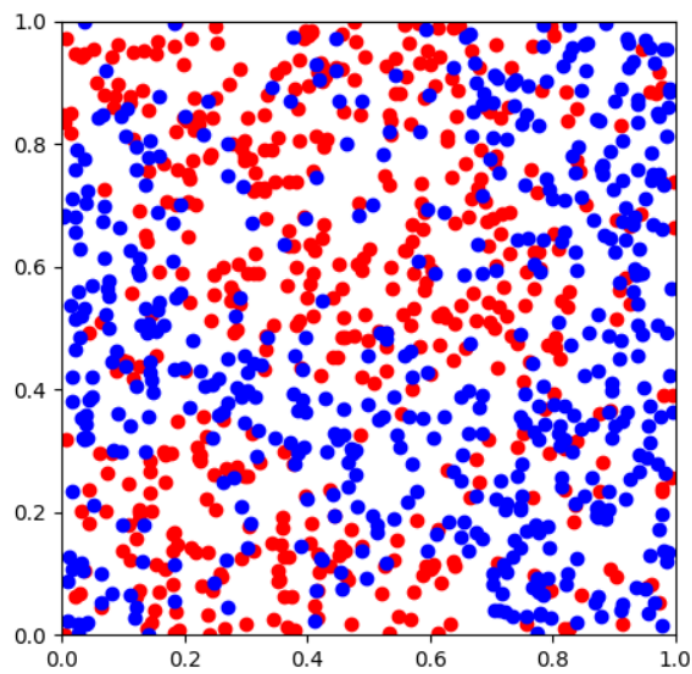
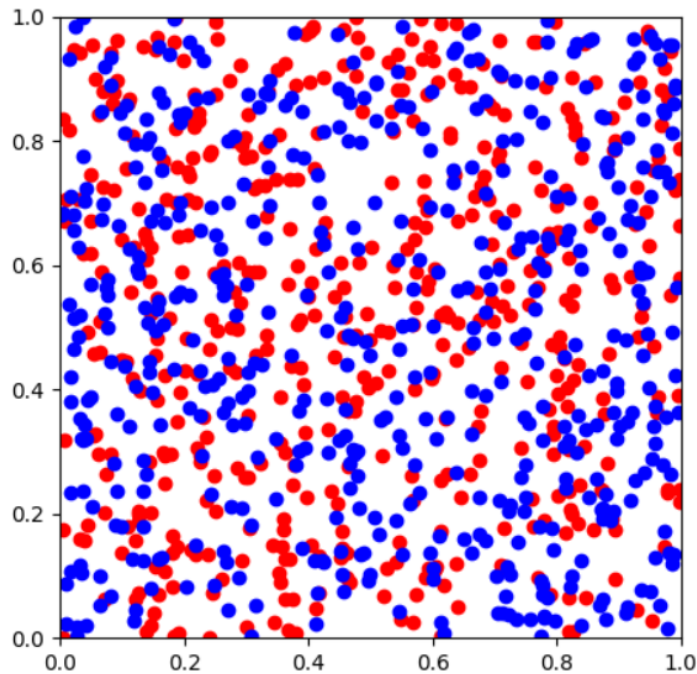
```
neighbors = [nb for nb in agents
              if (ag.x - nb.x)**2 + (ag.y - nb.y)**2 < r**2 and nb != ag]
```

Example 9.1 –cont.

Results for radius $r=0.1$ and the threshold for moving $th = 0.5$.

Agents self-organize from an initially random distribution to a patchy pattern with two clearly segregated patterns, as on figures below.

Full code is uploaded on Loop page in week 9 under 'abm-segregation-continuous.py'.



Example 9.2. (upload your solution on Loop)

- Adapt example 9.1 - perform simulations of Schelling's segregation model with th (threshold for moving), r (neighborhood radius), and/or n (population size = density) varied systematically.
- Determine the condition in which segregation occurs.
- Is the transition gradual or sharp?

Example 9.3. (theoretical + your own code)

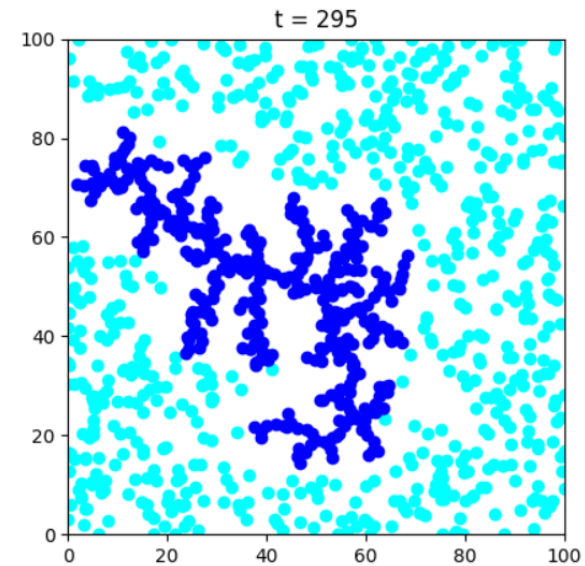
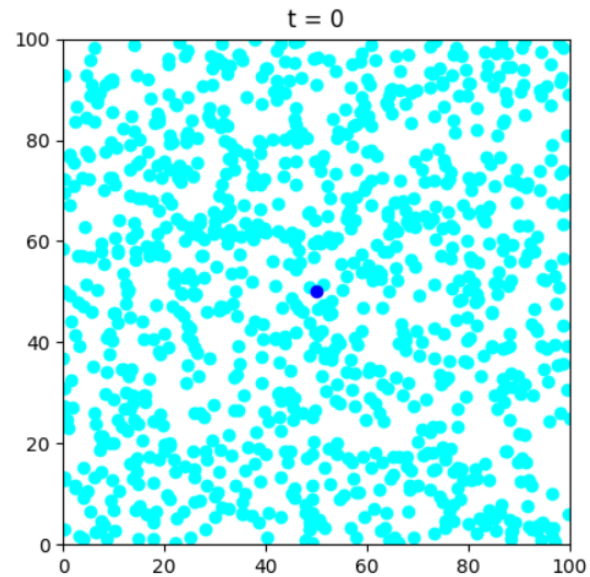
- Diffusion-limited aggregation model (DLA) – another historically important model.

Rules:

- Two types of particles, like in Schelling's segregation model, but only one of them can move freely.
- The movable particle diffuse in a 2D space by random walk, immovable particles do nothing; remain where they are.
- If a movable particle “collides” with an immovable particle (i.e., come close enough to each other), the movable particle becomes immovable and stays there forever.
- Conduct a simulation with all particles initially movable, except for one immovable “seed” particle placed at the center of the space, and observe what kind of spatial pattern emerges. Also carry out the simulations with multiple immovable seeds randomly positioned in the space, and observe how multiple clusters interact with each other at macroscopic scales.
- Completed Python simulator code of the DLA model is available on your Loop page under ‘9.2 *abm-DLA.py*’ but you are expected to try implementing your own simulator code to some extent first.

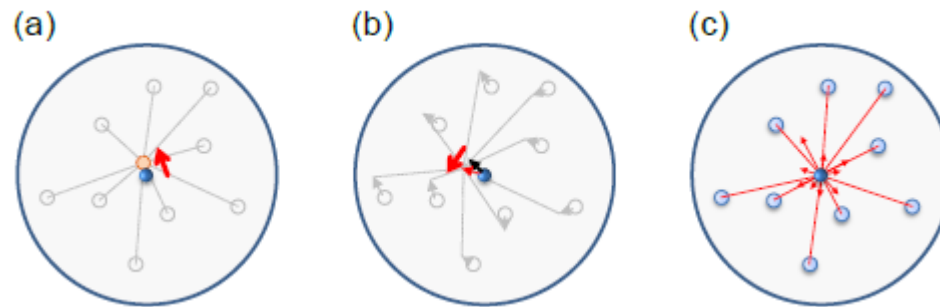
Example 9.3. (cont.)

- Diffusion-limited aggregation model (DLA) – an example of DLA output from the simulator



Example 9.4. (optional for labs)

- **Boids**
- Based on rules for Boids, explained in lectures earlier on.
- Main Boids rules are: cohesion, alignment, separation
- Collective behavior of animal groups, such as bird flocks, fish schools, and insect swarms in later models follow similar logic.
- Design an ABM of collective behavior with these three rules, and implement its simulator code. Conduct simulations by systematically varying relative strengths of the three rules above, and see how the collective behavior changes.



Three essential behavioral rules of Boids. (a) Cohesion. (b) Alignment. (c) Separation.

Ecological and Evolutionary Models

- Such ABMs are different from the other examples discussed so far in one important aspect: Agents can be born and can die during a simulation.
- This means that the number of state variables involved in a system can change dynamically over time, so the traditional concepts of dynamical systems don't apply easily to those systems.
- Simulating an ABM with a varying number of agents requires special care for simulating births and deaths of agents.
- Agents can be added to or removed from the system at any time during an updating process, so tricky to implement synchronous updating of agents. It is much more straightforward to update the system's state in an asynchronous manner.

Example 9.5. (theoretical)

- **Predator – prey model extension** An illustrative example of ecological ABMs
- The basic idea of this model is quite simple:
 - Prey naturally grow but get eaten by predators, while the predators grow if they get prey but otherwise naturally die off.
- Ecological dynamics should be described at an individual agent level, not at an aggregated population level. In the **six design tasks**:

1. *Design the data structure to store the attributes of the agents.*

The predator-prey ecosystem is obviously made of two types of agents: prey and predators. Note that attributes are identical to those of the agents in Schelling's segregation model, so we can use the same agent design:

```
r_init = 100 # initial rabbit population
f_init = 30 # initial fox population

class agent:
    pass

def initialize():
    global agents
    agents = []
    for i in xrange(r_init + f_init):
        ag = agent()
        ag.type = 'r' if i < r_init else 'f'
        ag.x = random()
        ag.y = random()
        agents.append(ag)
```

Example 9.5. (theoretical) – cont.

- Similar as earlier we can ignore 2, 3, 4 here as we are doing the simplest examples where the ABM doesn't involve an environment explicitly

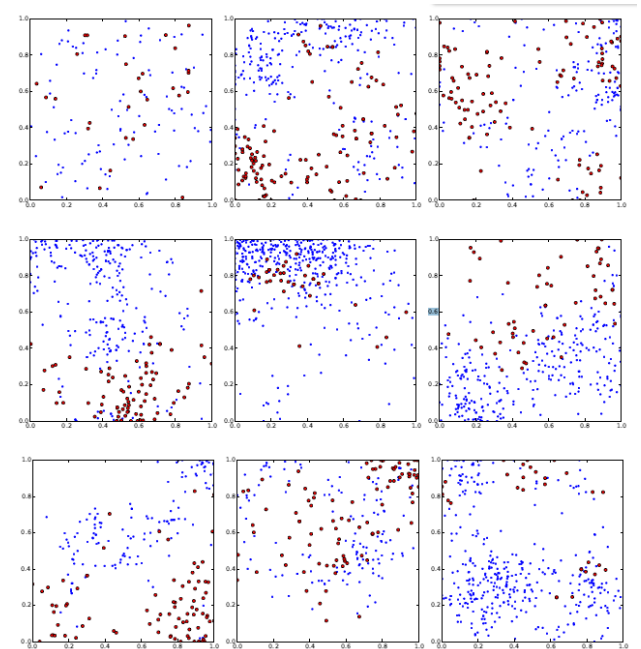
5. Describe the rules for how agents behave on their own, & 6. Describe the rules for how agents interact with each other.

In this model, the agents' inherent behaviors and interactions are intertwined.

- For prey agents, each individual agent reproduces at a certain reproduction rate. In ABMs, exponential growth means exponential increase of memory use because each agent physically takes a certain amount of memory space in your computer. We need to prevent such growth of memory use by applying a logistic-type growth restriction.
- In the meantime, if a prey agent meets a predator agent, it dies with some probability because of predation. Death can be implemented simply as the removal of the agent from the agents list.
- For predator agents, the rules are somewhat opposite. If a predator agent can't find any prey agent nearby, it dies with some probability because of the lack of food. But if it can consume prey, it can also reproduce at a certain reproduction rate.
- Finally, both types of agents diffuse in space by random walk. The rates of diffusion can be different between the two species, so let's assume that predators can diffuse a little faster than prey.
- The assumptions designed above can be implemented altogether in the *update* function.
- As you can see, the code is getting a bit longer than before, which reflects the increased complexity of the agents' behavioral rules
- *ag* is the agent randomly selected for *asynchronous* updating
- For full code check '9.2 abm-predator-prey.py' on your Loop under week 9

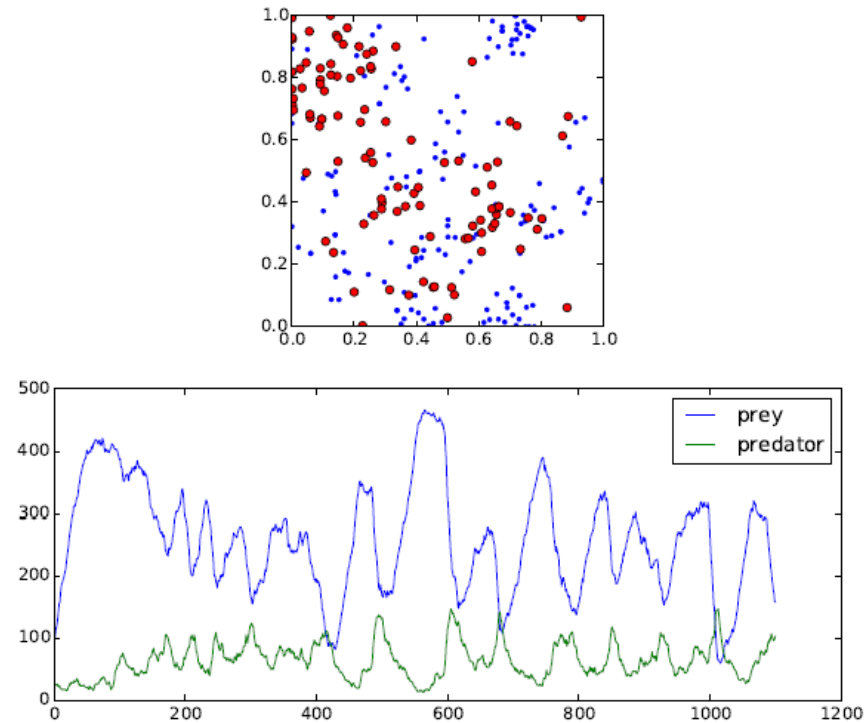
Example 9.5. (theoretical) – cont.

- **Predator – prey model results**
 - E.g. prey “rabbits” and predators “foxes” and their evolution and interaction
-
- The interplay between prey and predators produces very dynamic spatial patterns.
 - The prey population grows to form clusters (clouds of tiny blue dots), but if they are infested by predators, the predators also grow rapidly to consume the prey, leaving a deserted empty space behind.
 - As a result, the system as a whole begins to show dynamic waves of prey followed by predators. It is clear that spatial distributions of those species are highly heterogeneous.
 - The ABM generates complex dynamics that involve spatial locality and stochasticity. One could argue that the results obtained from this ABM version would be more realistic than those obtained from purely equation-based models.
 - Go to your Loop page and check the full code for ‘*abm-predator-prey.py*’



Example 9.5. (theoretical)

- **Predator – prey model – cont.**



- We can further modify the simulator code of the predator-prey ABM so that it outputs the time series plot of prey and predator populations next to the visualization of the agents' positions.

Example 9.6. (upload your solution on Loop)

- **Predator – prey model extension**
- In the current model settings in '9.2 abm-predator-prey.py' on your Loop page the predator-prey ABM occasionally shows extinction of predator agents (or both types of agents).
- How can you make the coexistence of two species more robust and sustainable?
- Develop your own strategy (e.g., adjusting model parameters, revising agents' behavioral rules, adding structures to space, etc.), implement it in your simulator code, and test how effective it is.
- Find what kind of strategy works best for the conservation of both species.

Evolution of agents

Evolution is a simple but very powerful dynamical process by which a population of organisms may spontaneously **optimise** their attributes for their own survival. It consists of the following three components:

- **Inheritance** - Organisms reproduce offspring whose attributes are similar to their own.
- **Variation** - There is some diversity of organisms' attributes within the population, which primarily arises from imperfect inheritance (e.g., mutation).
- **Selection** - Different attributes cause different survivability of organisms (*fitness*).

When all of these three components are present, evolution can occur in any kind of systems, e.g. biological but also social, cultural, and informational.

In example of predator-prey ABM, the selection is already there (i.e., death of agents). Therefore, we need to implement inheritance and variation in the model. For example, we can let the diffusion rates of prey and predators evolve spontaneously over time. To make them evolvable, we need to represent them as heritable traits. This can be accomplished by changing the previous code – check the code on your Loop week 9 under '*abm-predator-prey-evolvable.py*'